26T1 Week03

DBC, Modelling, Tests & Debugging

Software Design & Architecture

Annelise

agenda

- Access Modifiers (yeah again)
- Design by Contract
- Domain Modelling
- IDE Programming
- Exceptional Conditions

Modifier	Class	Package	Subclass	Global
Public	✓	✓	✓	✓
Protected	✓	✓	✓	✗
Default	✓	✓	✗	✗
Private	✓	✗	✗	✗

access modifiers

design by contract

- A way of designing software using **clear, formal agreements**. Provides an interface for others to use; **guarantees correct and expected behaviour**.
- **Each method has a contract** between the caller (who uses it) and the method (that provides the service)
- This contract specifies:
  - **Preconditions** – what must be true before the method is called
  - **Postconditions** – what is guaranteed after the method finishes
  - **Invariants** – conditions that must always remain true for the object
    - Note: invariants don't need to hold during the function call, but the end state should be invariant.

example: *bird class*

Consider the following example. Why might this be **bad design**? Or, what **kind of problems** would we run into with this design?

bird class

- method *fly()* takes in one integer argument, height.
- **precondition**: height must be greater than 5 metres
- **postcondition**: bird is now flying at that height.

---

penguin class

- overrides the *fly()* method
- **new precondition**: can only accept a height of 0 metres
- **new postcondition**: nothing changes

- We have just **strengthened the preconditions**.
  - That is, not every potential input for Bird works for Penguin (in fact, none of them do)
- We have also **weakened the postconditions**, since Penguin has an outcome which Bird doesn't.

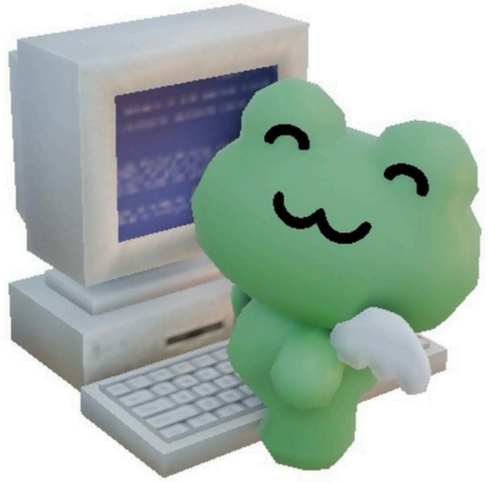
pre/postconditions

- Replacing a Bird with a Penguin → any calls to fly with an input of 5 metres will no longer be valid.
- Replacing a Bird with a Penguin → the object is no longer considered flying. Could break further code which relies on it flying

pre/postconditions

Liskov's Substitution Principle

"Objects of a **superclass should be replaceable** with objects of its subclasses **without breaking** the application or altering expected behavior."



code review

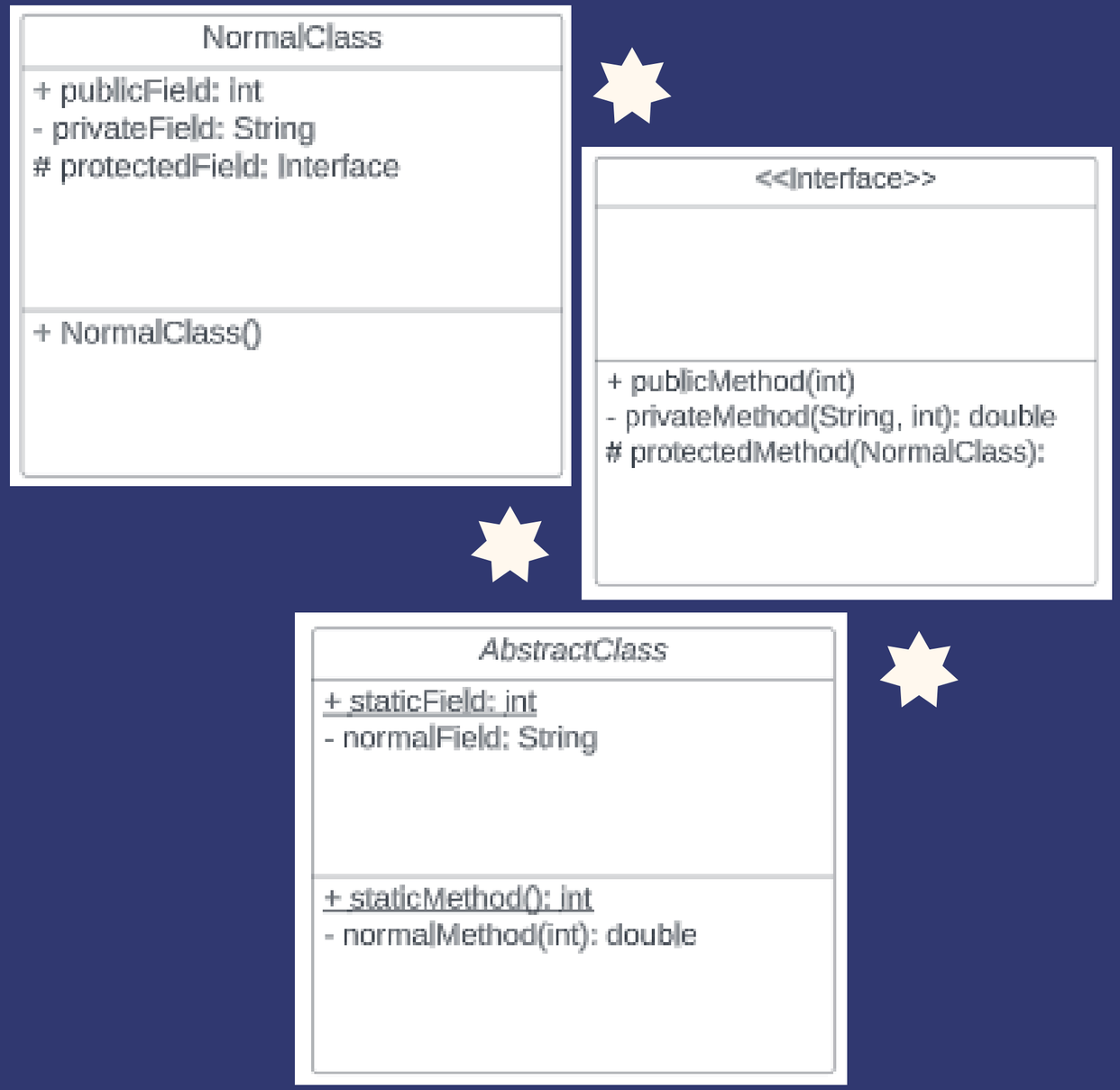
People

The process of turning real-world ideas into classes and objects in OOP, making software easier to understand and maintain by matching code to the problem it solves.

domain modelling

- Features both the **public and private fields and methods** for a class in the top half.
- Declares their **types** too (String, int, etc.)
- Normal classes have a constructor defined too.
- **Italicise**: abstract classes (title)
- **Underline**: static fields, static methods

UML Classes



1. Inheritance: "is-a" relationship

Denotes subclass → superclass

2. Realisation: "implements" relationship

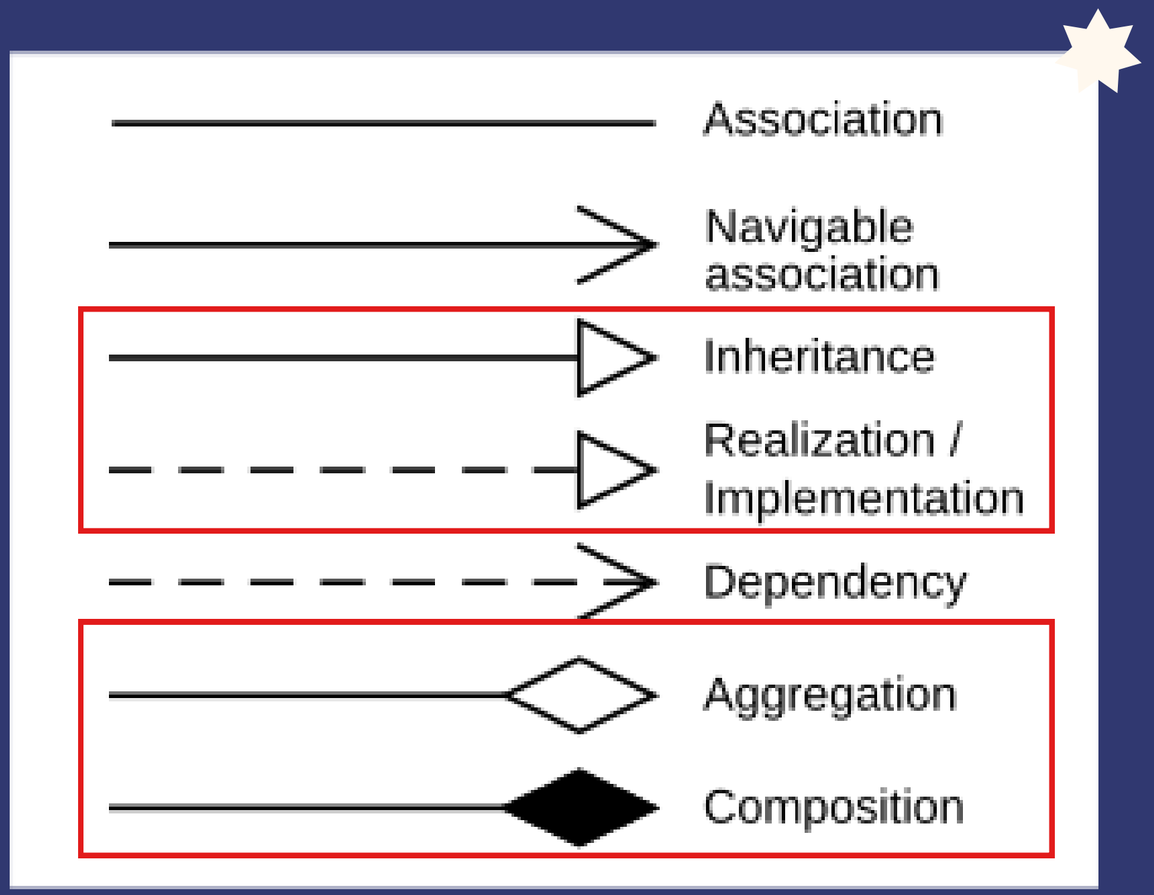
Denotes class → interface

3. Aggregation: "part-of" relationship

Class A aggregates an instance of B;
if an instance of A is destroyed
Instance of B can still exist

4. Composition: "has-a" relationship

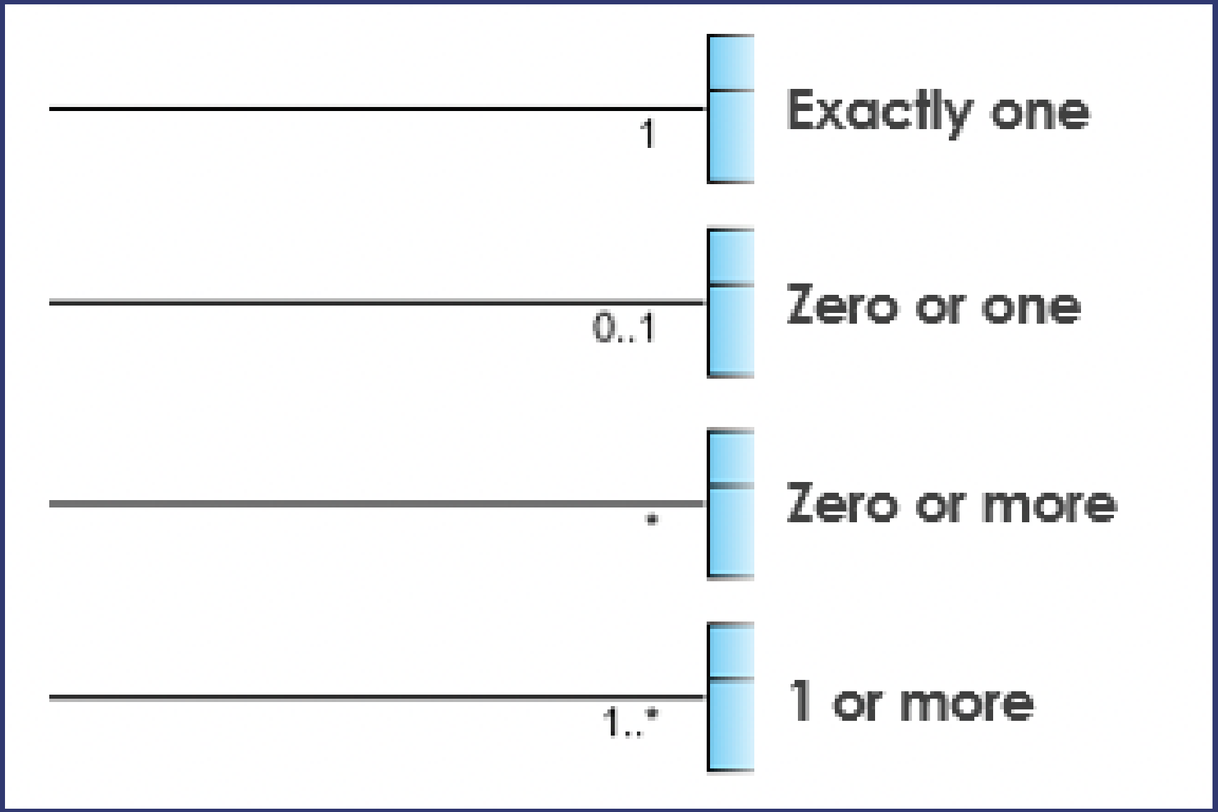
Class A composes of an instance of B;
if an instance of A is destroyed
Instance of B can no longer exist



UML Relationships

UML Cardinality

- Usually put this at the **end of both your arrows** as an annotation.
- They can have different types
  - one to one
  - one to many
  - many to many
- Here, we're saying a manufacturer can create zero or more Cars and a Car can only have one owner:



- A Car has **one or more engines and a manufacturer**. The manufacturing company **has a brand name**. Engines are also **produced by the manufacturer** and **have a speed**. There are only **two types of engines** within UNSW's cars:
  - **Thermal Engines**, which have a **default max speed of 114**, although they can be **produced with a different max speed**, and the max speed can change to any value between 100 and 250.
  - **Electrical Engines**, which have a **default max speed of 180.** This is the speed at which they are produced, and the max speed can change to any value that is divisible by 6.
- Cars are able to **drive to a particular location (x, y).**
- Since UNSW is a world-leader in technology innovation, they want you to be able to model the behaviour of Time Travelling for any vehicle, and to model a time travelling car. A vehicle that travels in time **stays in the same location but travels to a LocalDateTime.**

domain modelling

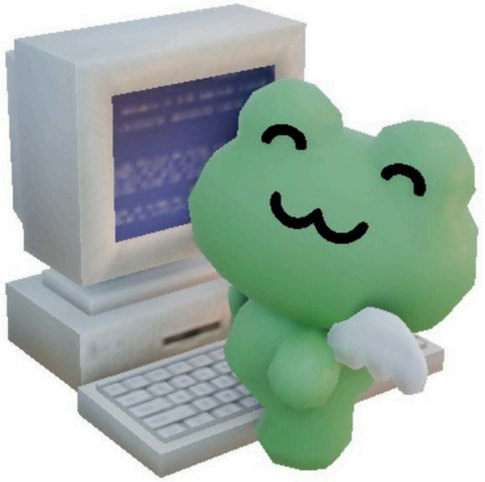
let’s model this!

LucidChart Demo

```java
You, 3 seconds ago | 1 author (You)
package wondrous.test;

import static org.junit.jupiter.api.Assertions.assertEquals;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import org.junit.jupiter.api.Test;

import wondrous.Wondrous;

You, 3 seconds ago | 1 author (You)
public class WondrousTest {
    @Test
    public void testBasic() {
        Wondrous w = new Wondrous();
        List<Integer> expected = new ArrayList<Integer>(Arrays.asL

        assertEquals(expected, w.wondrous(3));
    }      You, 2 minutes ago • initial commit …
}
```

JUnit Testing

- Popular Java testing framework
- Uses annotations like **@Test to mark test methods**.
- Provides **assertions to compare expected vs. actual results** (e.g. assertEquals).
- Supports **setup/teardown** with **@BeforeEach** and **@AfterEach**.
- Encourages test-driven development (TDD), improves code reliability.



code demo!

Wondrous

- The Wondrous Sequence is generated by the simple rule:
  - If **current term is even**, the next term is half the current term.
  - If **current term is odd,** the next term is three times the current term, plus 1.
- For example, the sequence generated by starting with 3 is:
  - 3 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1
- If the **starting term is 1, then an empty list** is returned.
- Wondrous.java has an implementation of this algorithm, but its test in WondrousTest.java is **failing!**

wondrous ✦

1. Use the debug tools to determine why the test is failing, and fix the bug.
2. There's another bug not caught by the given unit test. Find it and write a corresponding unit test.
3. Modify the method such that if start is less than 1, an IllegalArgumentException is thrown. Write a corresponding test for this inside WondrousTest.

wondrous ✦

exceptions

- an (undesirable) event that can occur during the execution of a program, and could **disrupt the normal flow** of that program.

1. Exception object is created and given to the runtime system (‘**throwing an exception**’)
2. Program execution is **paused**
3. The Java runtime will s**earch in the call stack for a method that can handle the exception**.

exceptions

- It's not always necessary to handle exceptions, since there are **both checked and unchecked exceptions** in Java.
- **Checked exceptions must be done manually** and will result in compilation error if not handled.
    - Anything that inherits from Exception class
- **Unchecked exceptions** are errors that occur at runtime.
    - Anything inheriting from RuntimeException

1. ~~Use the debug tools to determine why the test is failing, and fix the bug.~~
2. ~~There's another bug not caught by the given unit test. Find it and write a corresponding unit test.~~
3. Modify the method such that if start is less than 1, an IllegalArgumentException is thrown. Write a corresponding test for this inside WondrousTest.

wondrous ✦

lab time!

SitarME306